

Le dernier kilomètre se localise, il ne se franchit pas : une machine vérifiée par noyau qui nomme ce qui lui manque

Karl Olivet*

Préprint — août 2026

Traduction française de travail — la version de référence est l'anglaise (`main.tex`).

Résumé

Une recherche de preuve qui échoue ne rend en général rien qu'un ingénieur puisse utiliser. Nous rapportons un système où l'échec rend *la liste des formules qui fermentaient le but si on les ajoutait au pool* — un objet, pas un statut. Autour de cette primitive, vingt et un organes de recherche se sont accumulés en quatre mois, et la manière dont ils se sont accumulés est la revendication principale de cet article : **aucun ne vient d'une intuition d'architecte**. Chacun répond à un échec rejouable, et la colonne la plus informative du catalogue n'est pas ce que fait un organe mais quel diagnostic l'a produit. La trajectoire qui s'en dégage n'était pas prévue. Jusqu'à un certain point le système *chaîne* ce qu'on lui donne ; puis il accepte des témoins *proposés* ; puis il en *fabrique* un, en se servant du τ de Bourbaki pour bâtir un terme canonique depuis le seul but — et le passage a été forcé par un manque que la machine avait écrit elle-même, $\neg(n = n)$. Nous rapportons aussi ce que l'instrument rapporte ailleurs : étant donné une opération inventée le matin même, $a \oplus b := (a + b) + 1$, et deux lois brutes sur $+$, trois organes établissent sa commutativité en moins de deux secondes et son associativité en 16,43 s, closes par le noyau. Deux résultats négatifs constituent le bénéfice pratique. Une recherche qui échoue *en grossissant* — coût qui double par palier, nombre de manques figé — ne se répare presque jamais par plus de budget ; deux fois sur deux la cause était en amont. Et un corpus de pièges mesurés, quatorze, contient une loi de structure de ce noyau qu'aucune conception n'aurait pu anticiper : *les termes sont opaques, les formules se décomposent*. Enfin, et c'est le plus inconfortable, nous rapportons que notre propre documentation de ce travail avait dérivé sur huit chiffres sur huit, et ce que cela dit d'un article dont la thèse est qu'on mesure au lieu de décréter.

*Chercheur indépendant. Contact : karl.olivet@gmail.com. Code et corpus certifié : <https://github.com/Karl-0/bourbaki-theorie-des-ensembles>, au tag `preprint-v1 (54ceef5)`, qui épingle l'état exact sur lequel chacune des mesures rapportées ici a été prise.

1 Introduction

Demandez à un prouveur automatique un théorème hors de sa portée et il rend un échec : un délai dépassé, un budget épuisé, une liste vide. Le contenu informationnel de cette réponse est proche de zéro, et c'est la même réponse que le but ait été à un lemme près ou hors d'atteinte par principe. Tout ce que la recherche a appris en chemin — quels sous-buts se sont fermés, quelle hypothèse a refusé de se décharger, quelle branche est morte et pourquoi — est jeté avec lui.

Cet article rapporte un système bâti sur la convention inverse. Quand sa recherche échoue, il rend **l'ensemble des formules qui fermentaient le but si on les ajoutait au pool**. L'échec devient une donnée qui a une forme : non pas « non », mais « non, et voici exactement ce qui manque ». Nous appelons *organe de besoin* le composant qui fait cela.

Le dernier kilomètre, défini. Nous empruntons le terme à la logistique, où il nomme le dernier tronçon d'une livraison — court, démesurément coûteux, et celui qu'aucun investissement dans le réseau principal ne raccourcit. En recherche de preuve, il nomme ceci :

Étant donné un but, une recherche dérive ce qu'elle peut, puis s'arrête.

Le dernier kilomètre est la distance entre ce qu'elle a dérivé et ce qui était demandé — le tronçon où la recherche mécanisée a fait tout ce qu'elle sait faire et où le but reste ouvert.

Trois propriétés en font un problème à part entière, et non simplement « la partie difficile ». Il n'est *pas proportionnel à l'effort* : doubler le budget ne le divise pas par deux, parce que ce qui manque d'ordinaire n'est pas plus de recherche mais un objet d'une autre nature — un témoin que le pool n'a jamais nommé, une loi à une instance que personne n'a engendrée. Il n'est *pas visible* : un délai dépassé a la même tête qu'il manque un lemme ou une théorie entière. Et c'est *là que l'humain reprend la main* — ce qui est précisément pourquoi le laisser sans description coûte cher : la personne qui reprend le travail doit redécouvrir ce que la machine savait déjà et a jeté.

La thèse. Le dernier kilomètre ne se franchit pas d'un bond ; il se *localise*. Un système qui échoue en produisant l'énoncé exact de son manque vaut mieux qu'un système qui réussit une fois sur deux sans savoir pourquoi — parce que le premier s'améliore par construction, et que le second ne peut qu'être relancé.

Ce qui rend la revendication vérifiable. Une thèse sur la façon dont les outils devraient croître est facile à affirmer et difficile à contrôler. La nôtre est contrôlable parce que la croissance est consignée. L'organe s'est accru de vingt et une extensions en quatre mois, et pour chacune le dépôt garde l'échec qui l'a motivée, sous une

forme qui rejoue encore. Le catalogue de la section 4 porte donc deux colonnes, et **c’est celle de droite** — « **né de quel diagnostic** » — **qui porte la charge**. Si la thèse était fausse, cette colonne se lirait comme une rationalisation écrite après coup. Ce n’est pas le cas : chaque entrée nomme une mesure.

Le cadre. Tout tourne contre un noyau de style LCF implémenté sur le formalisme propre de Bourbaki, décrit dans un article compagnon. Deux de ses propriétés comptent ici. L’oracle est *exact* : un témoin proposé ne coûte au pire qu’une route morte, puisque le noyau juge — le système peut donc se permettre de deviner. Et le quantificateur existentiel n’est pas primitif — $(\exists x)R$ abrège la substitution d’un τ -terme dans R — ce qui permettra plus tard à un organe de *bâtir* un témoin à partir d’un but au lieu d’en chercher un.

Contributions.

1. **L’échec comme donnée** (section 3). L’organe de besoin rend les obligations qui fermentaient le but, jugées par le noyau. 387 lignes, protégées par 20 tests.
2. **Une loi de croissance, et sa preuve** (section 4). Vingt et un organes, chacun traçable à un échec rejouable. *L’outil se déduit du diagnostic*. La liste est ouverte, non une taxonomie close.
3. **Une frontière tracée par la machine** (section 5). Chaîner, puis accepter des propositions, puis fabriquer. Le passage a été forcé par un manque que le système a écrit lui-même, et le gain qu’il apporte n’est *pas* quantitatif : le même nombre de buts se ferme, mais la forme du manque résiduel change.
4. **Une algèbre neuve en trois organes** (section 6). Commutativité de $\oplus$ en moins de 2 s, associativité en 16,43 s, depuis deux lois brutes sur $+$.
5. **Deux signatures d’échec exploitables** (section 7). Échouer *en grossissant* indique une cause en amont, non un budget insuffisant ; et un corpus de quatorze pièges mesurés, dont une loi de structure du noyau.
6. **Notre propre dérive, mesurée** (section 8). La documentation de ce travail avait vieilli sur huit chiffres sur huit. Nous le rapportons parce qu’un article sur le fait de nommer ce qui manque doit à son lecteur le cas où il a cessé de le faire.

Ce que cet article ne revendique pas. Aucune mathématique nouvelle : les organes manipulent des énoncés, et $\oplus$ est un jouet dont les deux lois sont des corollaires immédiats de celles de $+$. Aucune architecture d’agent : il n’y a ni planificateur, ni politique apprise, ni boucle d’entraînement. Et aucune généralité : vingt et un organes ont été taillés sur deux sujets, et rien ne suggère que le vingt-deuxième problème n’en exigera pas cinq de plus — ce que la thèse prédit d’ailleurs.

2 Le substrat : le pool, la route, le manque

Nous ne résumons que le nécessaire ; le noyau et sa frontière de confiance font l'objet de l'article compagnon.

Un *théorème* est un triplet : un ensemble fini d'hypothèses dont il dépend encore, une conclusion, et la règle du noyau qui l'a produit. Il est *clos* quand l'ensemble d'hypothèses est vide. Nous écrivons $\vdash R$ lorsque le noyau peut construire un théorème clos de conclusion R ; chaque $\vdash$ ci-dessous est témoigné par un objet que le dépôt sait rebâtir, jamais affirmé.

Un *but* est une relation à dériver. Un *pool* est un ensemble fini de théorèmes disponibles pour la recherche. Une *route* est une dérivation tentée du but, et le *résidu* d'une route est l'ensemble d'hypothèses du théorème auquel elle a effectivement abouti. Un but a en général plusieurs routes, et une même hypothèse peut être ouverte sur l'une et close sur l'autre — tout verdict ci-dessous porte donc sur une hypothèse *sur une route*.

Définition 1 (Manque). *Un manque d'une route est une formule qui, ajoutée au pool, permettrait à cette route de fermer son but. L'organe de besoin rend, pour une recherche échouée, un ensemble fini de manques.*

La définition est délibérément faible : un manque n'est prétendu ni minimal, ni nécessaire, ni unique. C'est ce dont la recherche aurait eu besoin *sur la route qu'elle a prise*. Cette modestie est ce qui le rend calculable et, comme le montre la section 5, ce qui rend sa *forme* informative même quand sa taille ne l'est pas.

3 L'organe de besoin : un échec qui a une forme

L'organe prend un but et un pool, et rend soit une preuve certifiée par le noyau, soit une liste de manques. Il fait 387 lignes de Python et est protégé par 20 tests dans `test_autonomie.py` ; la suite complète de découverte est de **42 tests verts en 32 min 50 s** (section 9).

Deux propriétés rendent l'objet rendu utile plutôt que décoratif.

Le noyau juge, donc deviner est bon marché. Chaque candidat que l'organe essaie est soumis au noyau. Une mauvaise supposition ne produit aucun théorème et coûte une route morte. C'est ce qui rend abordable l'architecture de proposeurs de la section 5 : le système peut fabriquer un terme sans la moindre garantie, puisque rien en aval ne le croira sur parole.

Les manques sont les mots de la recherche elle-même. Ce ne sont pas une reformulation du but à l'usage d'un humain ; ce sont des formules que la recherche

aurait consommées. Cette distinction est ce qui les rend actionnables — et ce qui en fait une *mesure* de la distance restante plutôt qu’une estimation.

4 Le catalogue, et sa loi de croissance

Vingt et un organes ont été identifiés. Nous l’écrivons ainsi, plutôt que « il y en a vingt et un », parce que le catalogue est un relevé de rencontres et non une partition démontrée exhaustive : un organe s’ajoute quand un échec en réclame un, la liste croît donc en rencontrant de nouveaux échecs, et rien ici ne borne combien il en reste.

La revendication. Aucun des vingt et un n’a été conçu d’avance. Chacun répond à un échec qu’on peut rejouer. Quatre exemples, choisis parce que leurs diagnostics sont de natures différentes :

organe	ce qu’il fait	né de quel diagnostic
v4	instancie les faits- $\forall$ du pool sur le but	l’hypothèse de récurrence $S\{n\}$ restait <i>inerte</i> dans le pool : présente et jamais utilisée
v9	ferme un but $t = t$ par réflexivité	mesuré : une route traînait $2k = 2k$ comme <i>manque</i>
v11	témoins tirés des variables libres des faits du pool	v10 n’extrayait que les termes de tête ; un prédicat défini se déploie et <i>enfouit</i> son argument
v18	instancie une loi du pool au moment de l’appliquer	v17 cherchait les membres gauches <i>lit-téralement</i> : le pool dit $a + b$, le but contient $(a+b)+1$ — la loi était bonne, son <i>instance</i> manquait

Lisez la colonne de droite et celle de gauche devient presque dérivable. Cette asymétrie *est* la thèse : l’outil se déduit du diagnostic. Une architecture pensée d’avance aurait produit une autre table — une dont la colonne de droite dirait « pour la généralité » ou « pour la complétude », ce à quoi ressemble une rationalisation.

Remarque 1 (un organe disparu). Le catalogue liste un v3, « fusionner les manques de la voie directe ». Le chercher dans le code ne rend rien : il a été absorbé ou supprimé, et le catalogue n’a jamais été mis à jour. Nous laissons l’observation ici plutôt que de renuméroter discrètement, parce que la section 8 traite exactement de cela.

5 Chaîner, proposer, fabriquer

L'histoire de l'organe a trois régimes, et le système est passé de l'un à l'autre sous la pression plutôt que par décision.

Chaîner. Jusqu'à v9, l'organe combine ce qu'on lui donne : il instancie, détache, décompose, recompose. Face à un but existentiel et à un pool qui ne nomme aucun objet convenable, il rapporte le but lui-même comme manque — correct, et inutile.

Proposer. v6 fait de l'organe un point d'extension : les *proposeurs* de témoins sont des fonctions passées en paramètre, et `besoin.py` ignore ce qu'elles sont. v10 est le premier générique — face à $(\exists x)\varphi$, les candidats sont les t des faits $t \in A$ du pool. Il ne sait rien d'aucun problème particulier.

Fabriquer. v10 et v11 *choisissent* parmi les objets que le pool nomme. Quand le pool n'en nomme aucun, ils sont muets. v13 *bâtit* le témoin canonique $\tau_x(\varphi)$ depuis le seul but, ce que le formalisme de Bourbaki rend légitime comme terme plutôt que comme astuce.

Ce qui a forcé le passage. Pas une revue de conception. Le système, cherchant sur un but de décomposition avec pour seul proposeur σ suggérant $p := n$, a écrit comme manque terminal

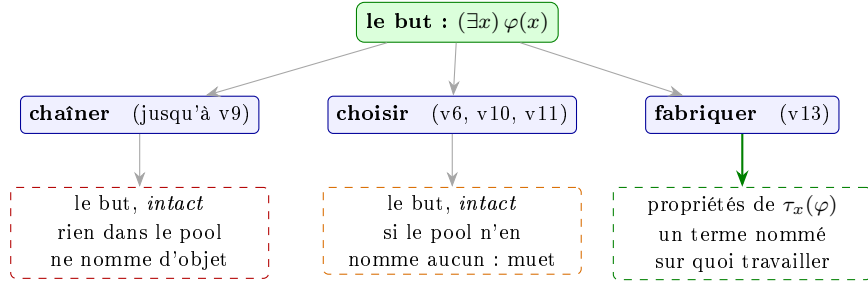
$$\neg(n = n).$$

Un manque qu'aucun pool ne peut combler : la recherche réclamait une contradiction parce qu'elle n'avait aucun moyen de nommer un autre objet. Lu comme une spécification — et c'en est une, puisque les manques sont des formules que la recherche consommerait — cela dit : *donne-moi un générateur de témoins*. v6 puis v13 sont la réponse à cette phrase.

Remarque 2 (le gain n'est pas quantitatif). v13 ne ferme pas plus de buts que v10. Jugé au nombre de manques produits, il aurait été écarté. Ce qui change, c'est leur *forme* : sur un but de décomposition à pool vide, v10 rend l'énoncé existentiel intact, et v13 rend les propriétés d'un terme nommé. Le système ne crée aucune information — il change la forme de la question. Mesurer le volume aurait masqué la seule chose qui s'est produite.

6 Une algèbre inventée le matin même, en trois organes

Les quinze premiers organes ont tous été taillés sur une seule conjecture. Une question différente a été posée le 11 août : la mécanique est-elle propre à ce sujet, ou sait-elle aborder une algèbre qu'elle n'a jamais vue ?



*le passage n'a pas été conçu : le manque terminal que
le système a écrit était $\neg(n = n)$ — lu comme une
spécification, « donne-moi un générateur de témoins »*

FIGURE 1 — Les trois régimes, et ce que chacun *rend quand il échoue* — la rangée du bas est la rangée informative. Chaîner et choisir rendent tous deux le but intact ; fabriquer rend les propriétés d'un terme nommé. Aucun but de plus ne se ferme (remarque 2) : ce qui change est la *forme* du résidu. **Cette figure illustre une distinction, elle ne mesure rien** — les temps des tests correspondants sont à la section 9.

Le protocole est volontairement nu. On définit une opération que le dépôt ne connaît pas,

$$a \oplus b := (a + b) + 1,$$

on donne au système *deux lois brutes* sur $+$ — associativité itérée, commutativité — et rien d'autre, puis on lui demande les propriétés de $\oplus$.

but	fermé par	mesuré
$a \oplus b = b \oplus a$	v16 (bâtit la congruence)	1,75 s
chaîne de réécritures	v17	3,40 s
$(a \oplus b) \oplus c = a \oplus (b \oplus c)$	v17 + v18	16,43 s

Les deux résultats sont clos, zéro hypothèse résiduelle. Les durées sont celles des tests correspondants et incluent donc les imports et la construction du pool — elles bornent le coût par le haut, ce qui est le sens qui compte ici.

La chaîne de l'associativité mérite d'être affichée, parce que c'est ce qu'un appa-
rieur littéral ne peut pas trouver et ce que v18 rend atteignable :

$$\begin{aligned}
 ((a + b) + 1) + c &= (a + b) + (1 + c) && \text{associativité itérée} \\
 &= (a + b) + (c + 1) && \text{commutativité, à l'intérieur de l'opérande droit} \\
 &= a + (b + (c + 1)) && \text{associativité de nouveau} \\
 &= a + ((b + c) + 1) && \text{et encore, dans l'autre sens}
 \end{aligned}$$

Cinq pas, dont aucun n'apparie une loi du pool *littéralement* : le pool dit $a + b$, le but contient $(a + b) + 1$. La loi était juste depuis le début ; ce qui manquait était son instance au sous-terme effectivement rencontré. C'est v18 en une phrase, et cela s'est trouvé en regardant un échec, non en lisant un traité de réécriture.

Ce que cela n'établit pas. $\oplus$ est un jouet, et ses deux lois sont des corollaires immédiats de celles de $+$. Rien de mathématiquement neuf n'est produit. Ce qui est acquis est plus modeste et plus solide : le système sait raisonner équationnellement sur une structure qu'il n'a pas vue, sans qu'on lui pré-mâche les instances. C'est un prérequis, pas un résultat.

7 Deux signatures d'échec exploitables

Échouer en grossissant. Le but d'associativité ci-dessus a d'abord échoué d'une manière qui ressemblait à un mur exponentiel : coût qui double à chaque cran de profondeur, et nombre de manques figé à un. La lecture naturelle est « le problème est dur, augmentons le budget ». Elle était fausse deux fois sur deux. Les causes réelles étaient en amont — un organe écrit deux fois, et une borne fixée au jugé plutôt qu'à la mesure : la recherche autorisait trois pas là où la chaîne minimale, affichée ci-dessus, en compte cinq.

Nous énonçons la règle qui en découle comme une signature, parce qu'elle est bon marché à reconnaître :

Une recherche dont *le coût explose pendant que son nombre de manques reste fixe* ne se répare presque jamais par plus de budget. Le coût qui croît signifie qu'elle énumère ; les manques qui ne bougent pas signifient qu'elle énumère la mauvaise chose. Regardez en amont : l'ordre d'exploration, ou une borne que personne n'a mesurée.

La conséquence pratique adoptée depuis est de *mesurer la longueur de chaîne attendue avant de régler la borne*, ce que la docstring concernée porte désormais en chiffres plutôt qu'en prose.

Des pièges, payés. Le dépôt garde quatorze pièges rencontrés et payés — non des bonnes pratiques recopiées ailleurs. Trois portent au-delà de ce projet :

- **Le test qui n'a jamais tourné.** Une conclusion (« le proposeur v12 ne ferme rien ») reposait sur un appel qui, faute d'un paramètre, était silencieusement retombé sur un défaut. *Asserter la capacité avant de la mesurer* ; une ligne d'introspection suffit. Chercher la raison a découvert le vrai défaut — les proposeurs ne franchissaient pas du tout une couche intermédiaire.
- **Le test que les deux branches passent.** Pour montrer que fabriquer dépasse choisir, le cas d'essai était fermé par les deux. Un test que les deux branches passent ne discrimine rien ; il faut construire exprès le cas où l'une échoue.
- **Les termes sont opaques, les formules se décomposent.** Dans ce noyau, $N(7)$ et $N(3) + N(4)$ sont tous deux des τ -termes à un seul argument. Il n'y

a rien où descendre : aucun évaluateur ne peut fonctionner en parcourant un terme. La seule voie est la reconstruction — bâtir le terme attendu et comparer — praticable parce que les assemblages sont hashables et l'égalité en $O(1)$. Les formules, elles, se décomposent normalement. La frontière n'est pas un choix de conception ; c'est un fait sur le substrat, et tout outil de ce genre doit s'y plier.

Remarque 3 (la même erreur deux fois en vingt minutes). Le troisième piège a été rencontré une fois sur les termes, puis aussitôt de nouveau sur la reconnaissance des prédicats, où le code naviguait par `.sous[0].sous[1].sous[0]`. La seconde fois, appliquer la loi qu'on venait d'écrire a marché du premier coup. La généralisation est plus brutale que la loi : *ne jamais supposer une structure — la regarder*. Trois lignes d'introspection ont réglé en quelques secondes ce que deux hypothèses successives avaient coûté.

8 Notre propre dérive, mesurée

Le matériau de cet article — un catalogue des organes et un corpus de pièges — a été écrit les 10–12 août. Avant d'en tirer une seule phrase, chacun de ses chiffres a été re-mesuré. **Huit sur huit avaient dérivé.**

ce que dit le document	mesuré le 20 août	nature
15 tests dans <code>test_autonomie.py</code>	20	tests ajoutés, document
24 verts, 40 min	31 dans le paquet ; 42 avec le corpus, 32 min 50	et <i>plus rapide</i> qu'annon
un catalogue de « dix-neuf lignes »	21 organes	le document se contred
<code>besoin.py</code> , 259 lignes	387	+128
l'organe v3	aucune trace en code	absorbé ou supprimé, ja
v16 : 0,29 s	1,75 s	<i>non comparable, voir ci</i>
v18 : 8 s	16,43 s	<i>idem</i>
v15 : 102 s → 0 s	test entier en 2,87 s	<i>idem</i>

La réserve qui compte. Les trois dernières lignes ne montrent pas que le document se trompait. `pytest -durations` chronomètre le *test entier* — imports, construction du pool, les deux passes — là où les chiffres du document chronométraient manifestement *l'appel de l'organe seul*. Les deux séries ne mesurent pas la même chose, et écrire « le document se trompait » serait aussi négligent que la dérive elle-même. Ce qui est établi, et suffit à la section 6, est une borne supérieure : moins de 2 s, 16,43 s.

La seule revendication que nous retirons est celle de v15. Le catalogue rapporte une première passe à 102 s et une seconde à 0 s — un rapport frappant pour une section sur la capitalisation. Le test entier tourne aujourd'hui en 2,87 s : ce rapport ne peut donc pas être obtenu dans la configuration actuelle. Le compounding lui-même tient : le test affirme que le proposeur appris seul, sans aucun accès à

l'arithmétique, referme le but. *Son amplitude, non.* Nous rapportons l'assertion et abandonnons le nombre.

Une figure que nous ne tracerons pas. La pièce maîtresse du matériau était une courbe du nombre de manques décroissant au fil des sondes, $14 \rightarrow 8 \rightarrow 6 \rightarrow 4 \rightarrow 1$, chaque chute étiquetée par l'organe qui l'avait causée. Elle vient d'un journal de session et aucun script ne la rejoue. Elle n'est donc **pas dans cet article**. Une courbe qu'on ne peut pas reproduire n'a pas sa place dans un article dont la thèse est qu'on mesure au lieu de décréter ; ouvrir dessus aurait été une réfutation en figure 1.

Pourquoi cette section existe. Il aurait été facile de re-mesurer en silence, d'imprimer les nouveaux chiffres et de ne rien dire. Nous rapportons la dérive parce qu'elle est la thèse de l'article retournée contre lui. La revendication est qu'un système doit nommer ce qui lui manque ; la documentation de ce système avait cessé de le faire, et aucun test ne l'a vu — le noyau certifie des dérivations, pas la prose écrite à côté. La même classe de défaut, à un autre niveau, fait l'objet de l'article compagnon sur la fidélité.

9 Reproductibilité

Une commande, un chiffre :

```
python -m pytest outils_ia/decouvertes/ tests/outils_ia/corpus/ -q
-durations=0
```

42 tests verts en 32 min 50 s, exit 0, au tag **preprint-v1** : 20 dans `test_autonomie.py`, 9 dans le sous-paquet d'autonomie, 2 sur des lemmes redécouverts, et 11 sur les deux organes les plus récents, qui vivent dans un autre arbre. Les verdicts se lisent sur la dernière ligne du fichier de sortie, jamais sur une notification : trois « exit 0 » de ce projet se sont avérés être des délais dépassés.

Le coût est concentré en un point. `test_euclide_infinitude` prend **1 361,87 s** — **69 % de toute la suite** ; chaque test d'organe est sous 17s. Protéger vingt et un organes est bon marché ; démontrer Euclide ne l'est pas.

10 Travaux voisins

La sélection de prémisses — le voisin le plus proche, et le contraste le plus net. Étant donné un but et une grande bibliothèque, la sélection de prémisses classe les faits existants à fournir au prouveur : par traits sémantiques construits

à la main [7], par classifieurs appris [1], par transformeurs sur la bibliothèque [10]. La question ressemble à la nôtre — *que faudrait-il ajouter ?* — et la différence est exactement là où gît l'intérêt. La sélection de prémisses *classe ce qui existe* ; l'organe de besoin *nomme des formules qui peuvent ne pas exister du tout*. Quand notre système rapporte $\neg(n = n)$ (section 5), aucun classement sur aucune bibliothèque n'aurait pu produire cette réponse, parce que la réponse n'est pas un élément de bibliothèque — c'est un énoncé sur ce qu'il aurait fallu donner à la recherche. L'une optimise une recherche documentaire ; l'autre caractérise une absence.

La réparation de preuves. Une deuxième famille prend une preuve qui *fonctionnait* et la répare après un changement de contexte, soit en adaptant l'automatisation au changement [14], soit en apprenant sur un corpus de réparations [12, 15]. L'entrée est un artefact cassé ; la nôtre est une recherche qui n'en a jamais produit. Les deux sont complémentaires — la réparation présuppose une preuve, et le dernier kilomètre est précisément là où il n'y en a pas.

L'échec comme donnée. Que l'activité de preuve jetée mérite d'être capturée a été défendu et instrumenté [13], et les systèmes actuels consomment les échecs comme négatifs d'entraînement ou comme invites de réparation [4]. Nous partageons la prémisses et différons sur l'objet. Dans ces systèmes, l'échec enregistré est un artefact *non vérifié* : un message, une trajectoire, le diagnostic qu'un modèle porte sur lui-même. Ici c'est un ensemble de formules que le noyau jugera si quelqu'un tente de s'en servir. C'est ce qui rend un manque actionnable plutôt que simplement consignable.

Guider la recherche, contre décrire son bord. Le guidage appris — ENIGMA dans un prouveur par saturation [5], des politiques sur des arbres de tactiques [16, 11], la recherche en hyper-arbre [8] — vise le même mur par l'autre côté : il cherche à faire réussir la recherche plus souvent. Rien de tout cela ne dit quoi que ce soit quand la recherche échoue quand même, ce qui est le cas dont traite cet article. Les deux sont orthogonaux, et un système qui aurait les deux vaudrait strictement mieux que le nôtre.

Exploration et conjecture. L'exploration de théories engendre des lemmes candidats et retient ceux qu'un prouveur confirme [6], et la synthèse récente catalogue ce que de tels systèmes proposent [17]. Nos organes engendrent des *obligations* plutôt que des conjectures : non des énoncés intéressants qui vaudraient d'être démontrés, mais les formules précises dont une route enlisée avait besoin. Les systèmes qui établissent quelque chose en échouant à démontrer — recherche de contre-exemples [2], réfutation apprise [9] — forment une troisième famille, et réfutent au lieu de localiser.

Croître par rencontre. Enfin, la forme de notre résultat — une bibliothèque d’organes accrue depuis des manques mesurés, chacun ayant mérité sa place — a une parente en synthèse de programmes, où un système fait croître une bibliothèque d’abstractions depuis les problèmes qu’il résout [3]. Le mécanisme diffère (compression là, diagnostic ici) mais la revendication épistémique est proche, et nous y voyons un indice que le motif n’est pas un artefact de notre cadre.

Nouveauté, énoncée honnêtement. Les objets d’échec formalisés ne sont pas neufs ; la sélection de prémisses non plus, ni la réparation. Ce que nous n’avons pas trouvé ailleurs est une recherche dont le mode d’échec est *une liste, vérifiable par le noyau, des formules qu’elle aurait consommées*, accompagnée d’un catalogue montrant que de tels objets, lus comme des spécifications, ont engendré les extensions de l’outil lui-même. Comme y insistent les articles compagnons, cette revendication est semi-décidable : aucun contre-exemple après recherche méthodique, jamais plus.

11 Limites

Deux sujets, vingt et un organes. Le catalogue a été taillé sur une conjecture et une algèbre jouet. Rien ici ne montre que les organes se généralisent, et la thèse elle-même prédit qu’un sujet vraiment neuf en réclamera d’autres. La lecture honnête de « vingt et un » est : ce que deux problèmes ont exigé.

Le catalogue est ouvert. Nous rapportons 21 organes identifiés, non une taxonomie démontrée exhaustive. Exhiber un vingt-deuxième est un acte fini ; prouver qu’il n’y en a pas ne l’est pas.

Les manques sont relatifs à une route, et non minimaux. La définition 1 ne prétend ni à la minimalité ni à la nécessité. Un manque est ce dont *la route prise* aurait eu besoin ; une autre route peut en rapporter d’autres, et aucun ensemble n’est canonique. L’utilité démontrée ici est pratique, non théorique.

Les durées bornent par le haut. Tous les temps sont des durées de test, décor inclus, mesurées une fois sur une seule machine Windows 11 (Python 3.13, pytest 9.0.3). Ce sont des bornes supérieures, non des mesures d’étalonnage, et aucune moyenne sur répétitions n’a été faite.

Revendications retirées ou non refaites. Le $102s \rightarrow 0s$ de v15 est retiré (section 8). Deux autres chiffres du matériau — un facteur 100 sur l’évaluation des termes, et un rapport $24\times$ sur la recherche d’associativité — n’ont pas été refaits ici ; ce sont des mesures d’époque et nous ne les présentons pas comme actuelles.

Un seul esprit. Conçu, exploité et audité par une seule personne. La re-mesure de la section 8 est un cas où l’auto-audit a fonctionné — mais il a fonctionné parce que le matériau allait être publié, non parce qu’un processus l’a attrapé.

12 Conclusion

Le système rapporté ici ne résout pas les problèmes sur lesquels on le pointe. Ce qu’il fait, c’est rendre, quand il échoue, l’énoncé de ce dont il aurait eu besoin — et cela a suffi à faire croître l’outil lui-même. Vingt et un organes, aucun conçu d’avance, chacun répondant à un échec qui rejoue encore ; une frontière entre chaîner et fabriquer que la machine a franchie sous la pression d’un manque qu’elle avait écrit ; et une algèbre inventée le matin, sur laquelle elle raisonne l’après-midi en moins de vingt secondes.

Les deux résultats négatifs sont ceux que nous garderions. Échouer *en grossissant* est une signature, et la lire correctement a sauvé un chantier que plus de budget n’aurait pas ouvert. Et la dérive de notre propre documentation, huit chiffres sur huit, est l’inconfortable : la discipline que cet article recommande pour une recherche de preuve est exactement celle que ses auteurs avaient cessé d’appliquer à leurs propres notes.

Un prouveur qui dit « non » n’apprend rien à personne. Un prouveur qui dit « non, et voici la formule qui m’a manqué » rend la seule chose qui fasse que la tentative suivante diffère de la précédente.

Références

- [1] Alexander A. Alemi, François Chollet, Niklas Een, Geoffrey Irving, Christian Szegedy, and Josef Urban. DeepMath — deep sequence models for premise selection. In *Proc. NeurIPS*, 2016.
- [2] Jasmin C. Blanchette, Lukas Bulwahn, and Tobias Nipkow. Automatic proof and disproof in Isabelle/HOL. In *Proc. FroCoS*, 2011.
- [3] Kevin Ellis, Catherine Wong, Maxwell Nye, et al. DreamCoder : Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proc. PLDI*, 2021.
- [4] Emily First, Markus Rabe, Talia Ringer, and Yuriy Brun. Baldur : Whole-proof generation and repair with large language models. In *Proc. ESEC/FSE*, 2023.
- [5] Jan Jakubův, Karel Chvalovský, Miroslav Olšák, Bartosz Piotrowski, Martin Suda, and Josef Urban. ENIGMA anonymous : Symbol-independent inference guiding machine. In *Proc. IJCAR*, 2020.
- [6] Moa Johansson, Dan Rosén, Nicholas Smallbone, and Koen Claessen. Hipster : Integrating theory exploration in a proof assistant. In *Proc. CISM*, 2014.

- [7] Cezary Kaliszyk, Josef Urban, and Jiří Vyskočil. Efficient semantic features for automated reasoning over large theories. In *Proc. IJCAI*, 2015.
- [8] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving. In *Proc. NeurIPS*, 2022.
- [9] Zenan Li, Zhaoyu Li, Kaiyu Yang, Xiaoxing Ma, and Zhendong Su. Learning to disprove : Formal counterexample generation with large language models. arXiv :2603.19514, 2026.
- [10] Maciej Mikula et al. Magnushammer : A transformer-based approach to premise selection. In *Proc. ICLR*, 2024.
- [11] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. arXiv :2009.03393, 2020.
- [12] Tom Reichel, R. Henderson, Andrew Touchet, Andrew Gardner, and Talia Ringer. Proof repair infrastructure for supervised models : Building a large proof repair dataset. In *Proc. ITP*, 2023.
- [13] Talia Ringer, Alex Sanchez-Stern, Dan Grossman, and Sorin Lerner. REPLica : REPL instrumentation for Coq analysis. In *Proc. CPP*, 2020.
- [14] Talia Ringer, Nathaniel Yazdani, John Leo, and Dan Grossman. Adapting proof automation to adapt proofs. In *Proc. CPP*, 2018.
- [15] Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, Jarod Alper, and Vasily Ilin. Learning to repair Lean proofs from compiler feedback. arXiv :2602.02990, 2026.
- [16] Kaiyu Yang and Jia Deng. Learning to prove theorems via interacting with proof assistants. In *Proc. ICML*, 2019.
- [17] Jian Zhang and Si-Cheng Tan. Automated conjecturing and theorem finding : A survey. *Journal of Computer Science and Technology*, 41(1) :46–66, 2026. DOI 10.1007/s11390-026-6040-0.